

Lec24

File System

File System means working with files and folders from code.

Using Node.js, we can:

- Create files
- Read files
- Write files
- Update files
- Delete files
- Rename files
- Work with folders/directories

Node provides a built-in module called:

`fs`

Import it like this:

```
const fs = require('fs');
```

Think of your computer as a library.

- Folder = Bookshelf
- File = Book
- fs module = Librarian

The librarian can:

- Find books
- Add books
- Remove books
- Update books

Node.js fs works similarly.

Types of File Operations

Synchronous Operations

- Runs line by line
- Blocks execution until task finishes

Example:

```
const fs = require('fs');
```

```
const data = fs.readFileSync('demo.txt', 'utf-8');  
console.log(data);
```

```
console.log("Done");
```

Asynchronous Operations

- Non-blocking
- Other code runs while file operation happens

Example:

```
const fs = require('fs');
```

```
fs.readFile('demo.txt', 'utf-8', (err, data) => {  
  if (err) throw err;  
  console.log(data);  
});
```

```
console.log("Done");
```

Possible output:

Done

Hello Students

Common FS functions

Read File

```
fs.readFile('demo.txt', 'utf-8', (err, data) => {  
  console.log(data);  
});
```

Write File

```
fs.writeFile('demo.txt', 'Hello Class', (err) => {  
  if(err) throw err;  
  console.log("File Written");  
});
```

If file exists → overwrite

If not exists → create new

Append File

```
fs.appendFile('demo.txt', '\nNew Line Added', (err) => {  
  if(err) throw err;  
});
```

Adds content at end.

Delete File

```
fs.unlink('demo.txt', (err) => {  
  if(err) throw err;  
});
```

Rename File

```
fs.rename('old.txt', 'new.txt', (err) => {  
  if(err) throw err;  
});
```

Why File System Important?

Used in:

- Logging systems
- Databases
- Configuration files
- Uploading files
- Report generation

Example:

- Social media app storing images
- E-commerce app storing invoices

Destructuring in Javascript/Node.js

Destructuring means extracting values from:

- Arrays
- Objects

and storing them into variables easily.

Without Destructuring

```
const student = {  
  name: "Naman",  
  age: 22  
};
```

```
const name = student.name;  
const age = student.age;
```

Long method.

With Destructuring

```
const student = {  
  name: "Naman",  
  age: 22  
};
```

```
const { name, age } = student;
```

```
console.log(name);  
console.log(age);
```

Easy and clean.

Object Destructing

```
const car = {  
  brand: "BMW",  
  model: "X5"  
};
```

```
const { brand, model } = car;
```

Rename Variables

```
const student = {  
  name: "Naman"  
};
```

```
const { name: studentName } = student;
```

```
console.log(studentName);
```

Default Values

```
const user = {  
  name: "Naman"  
};
```

```
const { name, city = "Delhi" } = user;
```

```
console.log(city);
```

Output:

Delhi

Array Destructing

```
const colors = ["Red", "Blue", "Green"];
```

```
const [first, second] = colors;
```

```
console.log(first);
```

```
console.log(second);
```

Output:

Red

Blue

Skipping values

```
const nums = [10,20,30];
```

```
const [a, , c] = nums;
```

```
console.log(a, c);
```

Output:

10 30

Why Use Destructuring in Node.js?

Very useful in:

- APIs
- Express.js
- Database queries
- Function parameters

Dependency

Dependency = Something your project depends on.

Example:

Your app uses:

- Express
- MongoDB
- Axios

These are dependencies.

Dependency Graph Meaning

A dependency graph shows:

- Which package depends on what
- Relationship between modules

Example:

Your App uses:

`express`

But express internally depends on many packages.

Graph:

My App

|

express

|

body-parser

|

debug

Actual dependency graphs can be huge.

Example with require()

Suppose:

app.js

```
const math = require('./math');
```

math.js

```
const helper = require('./helper');
```

helper.js

```
module.exports = {};
```

Dependency graph:

app.js

|

math.js

|

helper.js

Circular Dependency Problem

Example:

a.js

```
require('./b');
```

b.js

```
require('./a');
```

Problem:

- a loads b
- b loads a
- can create unexpected behavior

This is called circular dependency.

Package.json Role

Dependencies are stored in:

```
{  
  "dependencies": {  
    "express": "^4.18.0"  
  }  
}
```

package-lock.json

Stores exact dependency tree.

Example:

- express version
- sub dependencies version

This ensures same installation everywhere.

Async File Operations

This is one of Node.js's biggest strengths.

Node.js is famous for:

- Async programming
- Non-blocking I/O

What is Async?

Async means:

Task starts now, finishes later.

Meanwhile program continues running.

Example:

- Download file
- Read database
- Fetch API

These take time.

Node handles them asynchronously.

Why Needed?

Suppose reading a large file takes 10 seconds.

If synchronous:

Wait 10 seconds → Then continue

Bad for server.

If asynchronous:

Start reading file

Continue other tasks

Return result later

Much faster.

Example

```
const fs = require('fs');

console.log("Start");

fs.readFile('demo.txt', 'utf-8', (err, data) => {
  if(err) throw err;
  console.log(data);
});

console.log("End");
```

Output:

```
Start
End
File Content\
```

Async with Promises

Modern approach:

```
const fs = require('fs').promises;  
  
fs.readFile('demo.txt', 'utf-8')  
  .then(data => console.log(data))  
  .catch(err => console.log(err));
```

Async with Async/Await

```
const fs = require('fs').promises;
```

```
async function readData() {  
  try {  
    const data = await fs.readFile('demo.txt', 'utf-8');  
    console.log(data);  
  } catch(err) {  
    console.log(err);  
  }  
}
```

```
readData();
```

Task

Perform read, write, update, and delete operations on files